Maven Plugins & Execution
Step-by-Step Practical Manual for Students

Covers Compiler Plugin, Surefire Plugin, Shade Plugin, and Maven Wrapper (mvnw)

Aim	To create, build, test, package, and run a Java Maven project using the Compiler, Surefire, and Shade plugins, and then rebuild it with Maven Wrapper.
Platform	macOS terminal steps are shown. The same Maven commands work on Windows/Linux with minor path changes.
Java Version	Java 17
Project Name	MyMavenApp
Final Output	A working runnable shaded JAR that prints: Sum = 30

Important correction before starting

Many students get a test error because the default Maven quickstart project creates an old JUnit-style AppTest.java. In this manual, that file is replaced with a JUnit 5 version. This is the correct setup for modern Maven projects.

Software required

- Java 17 installed and available in terminal.
- Maven installed temporarily for project creation and wrapper generation.
- VS Code or any code editor.
- Internet connection for downloading Maven dependencies the first time.

Quick learning map

- mvn compile -> Compiler plugin compiles source files
- mvn test -> Surefire plugin runs unit tests
- mvn package -> Shade plugin creates runnable fat/uber JAR
- ./mvnw clean package -> Maven Wrapper builds the project using wrapper scripts

Step 1 - Open Terminal

Press Command + Space, type Terminal, and press Enter.

You should see a prompt similar to:

```
(base) yourname@MacBook-Air ~ %
```

Step 2 - Move to Desktop

Type the following command and press Enter:

```
cd ~/Desktop
```

Step 3 - Check your current location

Run the following command to confirm where you are:

```
pwd
```

Step 4 - List available files and folders

Use this command to view current Desktop contents:

```
ls
```

Step 5 - Create the Maven project

Run this Maven archetype command exactly as written:

```
mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenApp
-DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

Expected result: Maven downloads archetype components if needed and ends with BUILD SUCCESS.

Step 6 - Enter the project folder

After BUILD SUCCESS, move inside the created project folder:

```
cd MyMavenApp
```

Step 7 - Check the created project files

Verify that Maven created the project structure:

```
ls
```

Step 8 - Open the project in VS Code

If VS Code is installed with the code command, run:

```
code .
```

Step 9 - Open and replace pom.xml

Delete the old content of pom.xml and paste this corrected version:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
```

```
<artifactId>MyMavenApp</artifactId>
<version>1.0-SNAPSHOT</version>
<name>MyMavenApp</name>

<properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <mainClass>com.example.App</mainClass>
    <junit.version>5.10.2</junit.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>${junit.version}</version>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.13.0</version>
            <configuration>
                <release>${maven.compiler.release}</release>
            </configuration>
        </plugin>

        <plugin>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.2.5</version>
        </plugin>

        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-shade-plugin</artifactId>
            <version>3.5.3</version>
            <executions>
                <execution>
                    <phase>package</phase>
                    <goals>
                        <goal>shade</goal>
                    </goals>
                    <configuration>
                        <createDependencyReducedPom>true</createDependencyReducedPom>
                        <transformers>
```

```
                                <transformer
implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTransformer">
                                    <mainClass>${mainClass}</mainClass>
                                </transformer>
                            </transformers>
                        </configuration>
                    </execution>
                </executions>
            </plugin>
        </plugins>
    </build>
</project>
```

Step 10 - Save pom.xml

Press Command + S to save the file.

This pom.xml does four things: sets Java 17, adds JUnit 5, configures Surefire, and makes a shaded executable JAR.

Step 11 - Replace App.java

Open src/main/java/com/example/App.java and paste this code:

```
package com.example;

public class App {
    public static void main(String[] args) {
        Calculator calculator = new Calculator();
        int result = calculator.add(10, 20);
        System.out.println("Sum = " + result);
    }
}
```

Step 12 - Create Calculator.java

Inside src/main/java/com/example, create a new file named Calculator.java and paste:

```
package com.example;

public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }

    public int multiply(int a, int b) {
        return a * b;
    }
}
```

Step 13 - Replace AppTest.java with the corrected JUnit 5 file

Open src/test/java/com/example/AppTest.java, delete the old content, and paste:

```
package com.example;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

public class AppTest {

    @Test
    void testAdd() {
        Calculator calculator = new Calculator();
        assertEquals(30, calculator.add(10, 20));
    }

    @Test
    void testMultiply() {
        Calculator calculator = new Calculator();
        assertEquals(50, calculator.multiply(10, 5));
    }
}
```

Step 14 - Save all source files

Press Command + S in every edited file.

At this stage, the project has one main class, one utility class, and one JUnit 5 test class.

Step 15 - Return to Terminal and confirm the project path

Go back to Terminal and run:

```
pwd
```

Step 16 - Compile the project

Run the compile phase:

```
mvn compile
```

Expected result: BUILD SUCCESS. The Compiler plugin compiles your Java source files into target/classes.

Step 17 - Verify compiled classes

Check the target folder and compiled class files:

```
ls target
ls target/classes/com/example
```

Step 18 - Run unit tests

Now execute the test phase using Surefire:

```
mvn test
```

Expected result: BUILD SUCCESS. The Surefire plugin discovers AppTest.java and runs both test methods.

Step 19 - Verify Surefire reports

After tests pass, inspect the generated test report folder:

```
ls target/surefire-reports
cat target/surefire-reports/com.example.AppTest.txt
```

Step 20 - Package the application

Run the package phase to create the executable shaded JAR:

```
mvn package
```

Expected result: BUILD SUCCESS. The Shade plugin creates a runnable fat JAR that includes required dependencies.

Step 21 - Check the generated JAR files

List the target directory again:

```
ls target
```

Step 22 - Run the shaded JAR

Execute the application directly from the JAR file:

```
java -jar target/MyMavenApp-1.0-SNAPSHOT.jar
```

Expected output: Sum = 30

Step 23 - Clean previous build output

Remove the old target folder:

```
mvn clean
```

Step 24 - Rebuild from a clean state

Now build everything again from scratch:

```
mvn clean package
```

Step 25 - Generate Maven Wrapper

Create wrapper scripts so students can build the same project without depending on a system-wide Maven setup:

```
mvn wrapper:wrapper
chmod +x mvnw
```

Step 26 - Build with Maven Wrapper

Finally, run the build with wrapper and test the JAR again:

```
./mvnw clean package
java -jar target/MyMavenApp-1.0-SNAPSHOT.jar
```

Expected output after running the JAR: Sum = 30

What each plugin did in this practical

Plugin / Tool	Command used	Purpose in this practical
maven-compiler-plugin	mvn compile	Compiles Java source code using Java 17.
maven-surefire-plugin	mvn test	Runs JUnit 5 unit tests from src/test/java.
maven-shade-plugin	mvn package	Creates one runnable shaded or fat JAR.
Maven Wrapper (mvnw)	./mvnw clean package	Builds the same project through wrapper scripts.

Most common student errors and fixes

Error: package junit.framework does not exist - Cause: the old quickstart AppTest.java was left unchanged. Fix: replace it fully with the JUnit 5 AppTest.java shown in Step 13.

Error: no main manifest attribute - Cause: missing Shade manifest transformer. Fix: keep the maven-shade-plugin configuration exactly as shown in Step 9.

Error: mvnw permission denied - Cause: wrapper script is not executable. Fix: run chmod +x mvnw.

Error: source option or release issue - Cause: Java version mismatch. Fix: confirm java -version and keep <maven.compiler.release>17</maven.compiler.release> in pom.xml.

Error: code command not found - Cause: VS Code shell command is not enabled. Fix: open VS Code manually and choose File > Open Folder.

One-shot command summary for teachers

```
cd ~/Desktop
mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenApp
-DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
cd MyMavenApp
mvn compile
mvn test
mvn package
java -jar target/MyMavenApp-1.0-SNAPSHOT.jar
mvn clean
mvn clean package
```

```
mvn wrapper:wrapper
chmod +x mvnw
./mvnw clean package
java -jar target/MyMavenApp-1.0-SNAPSHOT.jar
```

Result

The student successfully creates a Maven project, configures Compiler, Surefire, and Shade plugins, runs tests, packages the project into an executable shaded JAR, and rebuilds it through Maven Wrapper.